# Εθνικό Μετσόβιο Πολυτεχνείο

# Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

## 6η Εργαστηριακή Άσκηση

## Cross-compiling προγραμμάτων για ARM αρχιτεκτονική

**Μάθημα:** Σχεδιασμός Ενσωματωμένων Συστημάτων

**Εξάμηνο:** 9ο

**Ονοματεπώνυμα:** Μιχαλακόπουλος Δημήτριος (ΑΜ: 03120229), Γκενάκου Ζωή (ΑΜ: 03120015)

## ***Προετοιμασία για την άσκηση***

Για την ορθή πραγματοποίηση της άσκησης χρειάστηκε να πραγματοποιήσουμε τα παρακάτω βήματα:

1. Αρχικά, δημιουργήσαμε ένα δεύτερο εικονικό μηχάνημα (Virtual Machine) στο QEMU. Για να γίνει αυτό κατεβάσαμε και τοποθετήσαμε στον ίδιο φάκελο τα παρακάτω αρχεία:

   - https://people.debian.org/~aurel32/qemu/armhf/debian_wheezy_armhf_standard.qcow2
   - https://people.debian.org/~aurel32/qemu/armhf/initrd.img-3.2.0-4-vexpress
   - https://people.debian.org/~aurel32/qemu/armhf/vmlinuz-3.2.0-4-vexpress

2. Όπως και στην προηγούμενη εργαστηριακή άσκηση ξεκινήσαμε το εικονικό μας μηχάνημα εκτελώντας την παρακάτω εντολή:

   ```
   sudo qemu-system-arm -M vexpress-a9 -kernel vmlinuz-3.2.0-4-vexpress -initrd initrd.img-3.2.0-4-vexpress -drive if=sd,file=debian_wheezy_armhf_standard.qcow2 -append "root=/dev/mmcblk0p2" -net nic -net user,hostfwd=tcp:127.0.0.1:22223-:22
   ```

3. Έπειτα χρειάστηκε να ανανεώσουμε το αρχείο *sources.list*, για να έχουμε πρόσβαση στα πακέτα των repositories του Debian, και τις *λίστες πακέτων* (apt-get update), όπως κάναμε και για το προηγούμενο εικονικό μας μηχάνημα.

## ***Εγκατάσταση custom cross-compiler building toolchain crosstool-ng***

Για την εγκατάσταση του crosstool-ng ακολουθήσαμε, όπως μας δίνονται από την εκφώνηση, τα παρακάτω βήματα:

1. Κατεβάσαμε τα απαραίτητα αρχεία για το χτίσιμο του toolchain από το αντίστοιχο *github repository* στο host μηχάνημα εκτελώντας την ακόλουθη εντολή:

   ```
   :~$ git clone https://github.com/crosstool-ng/crosstool-ng.git
   ```

   Η εντολή δημιούργησε στο directory που την εκτελέσαμε ένα φάκελο με το όνομα *crosstool-ng*.

2. Μπήκαμε σε αυτό τον φάκελο και εκτελέσαμε την παρακάτω εντολή:

   ```
   :~/crosstool-ng$ ./bootstrap
   ```

3. Στην συνέχεια, δημιουργήσαμε δύο φακέλους στο HOME directory ως εξής:

   ```
   :~/crosstool-ng$ mkdir $HOME/crosstool && mkdir $HOME/src
   ```

4. Για να κάνουμε configure την εγκατάσταση του crosstool-ng εκτελέσαμε την παρακάτω εντολή:

   ```
   :~/crosstool-ng$ ./configure --prefix=${HOME}/crosstool
   ```

   Κατά την εκτέλεση της παραπάνω εντολής εμφανίστηκαν πολλά πακέτα που έλειπαν. Έτσι, μέσω του package manager και μετά από κάποιες αναζητήσεις στο διαδίκτυο εγκαταστήσαμε καθένα από αυτά ως εξής:
   - Για το flex εκτελέσαμε την εντολή sudo apt-get install flex
   - Για το makeinfo εκτελέσαμε την εντολή sudo apt-get install texinfo
   - Για το unzip εκτελέσαμε την εντολή sudo apt-get install unzip
   - Για το help2man εκτελέσαμε την εντολή sudo apt-get install help2man
   - Για το file εκτελέσαμε την εντολή sudo apt-get install file
   - Για το GNU awk εκτελέσαμε την εντολή sudo apt-get install gawk
   - Για το libtool εκτελέσαμε την εντολή sudo apt-get install libtool-bin
   - Για το bison εκτελέσαμε την εντολή sudo apt-get install bison
   - Για το curses εκτελέσαμε την εντολή sudo apt-get install libncurses5-dev

5. Έπειτα εκτελέσαμε την εντολή *make* και *make install* ως εξής:

   ```
   :~/crosstool-ng$ make && make install
   ```

6. Μέχρι εδώ έχει εγκατασταθεί το *crosstool-ng*. Έπειτα πήγαμε το installation path $HOME/crosstool/bin ως εξής:

```
:~/crosstool-ng$ cd $HOME/crosstool/bin
```

   Σε αυτό τον φάκελο κάναμε build τον cross compiler μας.

7. Εκτελέσαμε την εντολή:

```
:~/crosstool/bin$ ./ct-ng list-samples
```

   Εμφανίζεται μια λίστα με πολλούς συνδυασμούς αρχιτεκτονικών, λειτουργικών συστημάτων και βιβλιοθηκών της C που παρέχει το εργαλείο για να κάνουμε configure το build του cross-compiler που θέλουμε να παράξουμε για ένα συγκεκριμένο target machine. Εμείς επιλέξαμε την: arm-cortexa9_neon-linux-gnueabihf.

8. Για να παραμετροποιήσουμε το crosstool-ng για την συγκεκριμένη αρχιτεκτονική εκτελέσαμε την παρακάτω εντολή:

```
:~/crosstool/bin$ ./ct-ng arm-cortexa9_neon-linux-gnueabihf
```

9. Για οποιεσδήποτε αλλαγές με γραφικό τρόπο στα χαρακτηριστικά του preconfigured συνδυασμού target machine μπορούμε να εκτελέσουμε την παρακάτω εντολή:

```
:~/crosstool/bin$ ./ct-ng menuconfig
```

   Εμείς χρησιμοποιήσαμε την βιβλιοθήκη της C glibc, η οποία ήταν ήδη επιλεγμένη.

10. Αφού παραμετροποιήσαμε τον cross compiler, τον κάναμε build με την ακόλουθη εντολή:

```
:~/crosstool/bin$ ./ct-ng build
```

11. Αφού ολοκληρώθηκε η παραπάνω διαδικασία, δημιουργήθηκε ένα φάκελος με path: $HOME/x-tools/arm-cortexa9_neon linux-gnueabihf μέσα στον οποίο βρίσκεται ο υποφάκελος bin που περιέχει όλα τα εκτελέσιμα αρχεία του cross compiler μας.

Γενικά, κατά την εγκατάσταση του crosstool-ng δεν αντιμετωπίσαμε κάποιο ιδιαίτερο πρόβλημα πέρα από το βήμα 4. Σε εκείνη μόνο την περίπτωση χρειάστηκε απλά να κατεβάσουμε τα πακέτα που έλειπαν στον χρήστη με δικαιώματα root (sudo).

## ***Εγκατάσταση pre-compiled building toolchain linaro***

Για την εγκατάσταση του linaro ακολουθήσαμε, όπως μας δίνονται από την εκφώνηση, τα παρακάτω βήματα:

1. Κατεβάσαμε τα binaries του cross compiler στο host μηχάνημα από την παρακάτω διεύθυνση, αφού πρώτα δημιουργήσαμε και πήγαμε στο αντίστοιχο directory:

   ```
   :~$ mkdir ~/linaro && cd ~/linaro
   :~/linaro$ wget
   https://releases.linaro.org/archive/14.04/components/toolchain/binaries/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux.tar.bz2
   ```

2. Ανοίγοντας το parent directory του tarball που κατεβάσαμε (https://releases.linaro.org/archive/14.04/components/toolchain/binaries/), παρατηρούμε στο κάτω μέρος της σελίδας ότι τα binaries του cross compiler συμπεριλαμβάνουν τα παρακάτω στοιχεία:
   - Linaro GCC 4.8 2014.04
   - Linaro Newlib 2.1 2014.02
   - Linaro Binutils 2.24.0 2014.04
   - Linaro GDB 7.6.1 2013.10
   - A statically linked gdbserver
   - A system root
   - Manuals under share/doc/
   - The system root contains the basic header files and libraries to link your programs against.

3. Κάναμε extract τα αρχεία που κατεβάσαμε με την παρακάτω εντολή:

   ```
   :~/linaro$ tar -xvf gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux.tar.bz2
   ```

4. Τα binaries του cross compiler θα πρέπει να βρίσκονται στο πακέτο που κατεβάσαμε στον φάκελο:

   $HOME/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin.

Γενικά, κατά την εγκατάσταση του linaro δεν αντιμετωπίσαμε κανένα απολύτως πρόβλημα.

## ***Άσκηση 1:***

Ο σκοπός του πρώτου μέρους της άσκησης είναι να απαντήσουμε σε καθένα από τα παρακάτω ερωτήματα:

1. Στην 5η εργαστηριακή άσκηση χρησιμοποιήσαμε το image debian_wheezy_armel για το QEMU, ενώ σε αυτή την άσκηση επιλέξαμε το debian_wheezy_armhf. Τα δύο αυτά images αναφέρονται σε διαφορετικές αρχιτεκτονικές ενός ARM επεξεργαστή για το Debian OS. Η armel (arm little-endian) αρχιτεκτονική που χρησιμοποιεί το πρώτο image, αναφέρεται σε παλαιότερους 32-bit little-endian ARM επεξεργαστές, που είναι συμβατοί με το v5te instruction set. Αυτή η αρχιτεκτονική χειρίζεται τον υπολογισμό αριθμών κινητής υποδιαστολής σε λειτουργία συμβατότητας που επιβραδύνει την απόδοση, αλλά επιτρέπει την συμβατότητα με κώδικα γραμμένο για επεξεργαστές χωρίς μονάδες κινητής υποδιαστολής. Στην συνέχεια, η armhf (arm hard-float) αρχιτεκτονική που χρησιμοποιεί το δεύτερο image αναφέρεται σε νεότερους arm επεξεργαστές (ARMv7), που είναι συμβατοί με το Thumb-2 instruction set. Αυτή η αρχιτεκτονική εκμεταλλεύεται το floating-point unit (FPU) και παρέχει άμεση υποστήριξη για floating-point operations απευθείας στο hardware, αλλά δεν είναι συμβατή με παλαιότερες αρχιτεκτονικές.

2. Χρησιμοποιούμε την αρχιτεκτονική *arm-cortexa9_neon-linux-gnueabihf* επειδή είναι συμβατή με το σύστημα που έχουμε διαμορφώσει στο QEMU. Όπως φαίνεται και από το όνομά της, υποστηρίζει την εκτέλεση αριθμητικών πράξεων κινητής υποδιαστολής (floating-point operations) απευθείας στο hardware (hf = hard-float) και είναι συμβατή με το EABI που χρησιμοποιείται από το armhf. Ακόμη, σύμφωνα με τα δοκίμια της ARM, ο Cortex-A9 βασίζεται στον 32-bit πυρήνα Armv7-A και καλύπτει τις βασικές απαιτήσεις του armhf, όπως την υποστήριξη Thumb-2 και VFPv3-D16 για floating-point operations. Εάν διαλέγαμε διαφορετική αρχιτεκτονική από το list-samples κατά το configuration του crosstool-ng, θα είχαμε προβλήματα συμβατότητας και το εκτελέσιμο που δημιουργείται από το toolchain δεν θα μπορούσε να εκτελεστεί στο QEMU.

3. Στο βήμα 9 χρησιμοποιήσαμε την βιβλιοθήκη glibc της C για την υλοποίηση. Ο βασικός λόγος για αυτή την επιλογή ήταν η συμβατότητά της με την αρχιτεκτονική ARMv7, καθώς και το γεγονός ότι προτείνεται επίσημα από τη Debian για χρήση σε περιβάλλοντα cross-compiling. Η glibc θεωρείται μια σταθερή και ευρέως υποστηριζόμενη λύση, διασφαλίζοντας ότι το παραγόμενο εκτελέσιμο αρχείο θα λειτουργεί αξιόπιστα στο target σύστημα, ενώ παράλληλα παρέχει καλή απόδοση. Παρόλο που υπάρχουν ελαφρύτερες εναλλακτικές, όπως η uClibc, οι οποίες μειώνουν το μέγεθος των εκτελέσιμων αρχείων και εξοικονομούν μνήμη, αυτές μπορεί να μην αποδίδουν εξίσου καλά σε απαιτητικά σενάρια. Για τις ανάγκες της συγκεκριμένης άσκησης, η glibc ήταν η πιο λογική και ασφαλής επιλογή. Η βιβλιοθήκη που χρησιμοποιήθηκε φαίνεται και από την έξοδο της παρακάτω εντολής:

```
ldd -v $HOME/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-cortexa9_neon-linux-gnueabihf-gcc
```

```
jim@Dimitris:~$ ldd -v $HOME/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-cortexa9_neon-linux-gnueabihf-gcc
        linux-vdso.so.1 (0x00007ffd5dbb4000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2703553000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f2703740000)

        Version information:
        /home/jim/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-cortexa9_neon-linux-gnueabihf-gcc:
                ld-linux-x86-64.so.2 (GLIBC_2.3) => /lib64/ld-linux-x86-64.so.2
                libc.so.6 (GLIBC_2.35) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.15) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.7) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.14) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.3) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.32) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.33) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.36) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.25) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.34) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.11) => /lib/x86_64-linux-gnu/libc.so.6
                libc.so.6 (GLIBC_2.2.5) => /lib/x86_64-linux-gnu/libc.so.6
        /lib/x86_64-linux-gnu/libc.so.6:
                ld-linux-x86-64.so.2 (GLIBC_2.35) => /lib64/ld-linux-x86-64.so.2
                ld-linux-x86-64.so.2 (GLIBC_2.2.5) => /lib64/ld-linux-x86-64.so.2
                ld-linux-x86-64.so.2 (GLIBC_2.3) => /lib64/ld-linux-x86-64.so.2
                ld-linux-x86-64.so.2 (GLIBC_PRIVATE) => /lib64/ld-linux-x86-64.so.2
```

4. Χρησιμοποιώντας τον cross compiler του crosstool-ng από τα προηγούμενα ερωτήματα κάνουμε compile τον κώδικα του αρχείου phods.c τρέχοντας την παρακάτω εντολή:

```
$HOME/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-cortexa9_neon-linux-gnueabihf-gcc phods.c  -O0 -Wall -o phods_crosstool.out
```

Έπειτα, τρέχουμε τις παρακάτω εντολές και παίρνουμε τις αντίστοιχες εξόδους:

```
file phods_crosstool.out
```

```
jim@Dimitris:~$ file phods_crosstool.out
phods_crosstool.out: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter
 /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not stripped
```

```
readelf -h -A phods_crosstool.out
```

```
jim@Dimitris:~$ readelf -h -A phods_crosstool.out
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           ARM
  Version:                           0x1
  Entry point address:               0x104e0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          13640 (bytes into file)
  Flags:                             0x5000400, Version5 EABI, hard-float ABI
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         9
  Size of section headers:           40 (bytes)
  Number of section headers:         36
  Section header string table index: 35
```

```
Attribute Section: aeabi
File Attributes
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
  Tag_ARM_ISA_use: Yes
  Tag_THUMB_ISA_use: Thumb-2
  Tag_FP_arch: VFPv3
  Tag_Advanced_SIMD_arch: NEONv1
  Tag_ABI_PCS_wchar_t: 4
  Tag_ABI_FP_rounding: Needed
  Tag_ABI_FP_denormal: Needed
  Tag_ABI_FP_exceptions: Needed
  Tag_ABI_FP_number_model: IEEE 754
  Tag_ABI_align_needed: 8-byte
  Tag_ABI_align_preserved: 8-byte, except leaf SP
  Tag_ABI_enum_size: int
  Tag_ABI_VFP_args: VFP registers
  Tag_CPU_unaligned_access: v6
  Tag_MPextension_use: Allowed
  Tag_Virtualization_use: TrustZone
```

Η πρώτη εντολή μας δίνει πληροφορίες για τον τύπο του αρχείου που πήρε ως argument. Αρχικά, βλέπουμε ότι πρόκειται για ένα εκτελέσιμο αρχείο (ELF) αρχιτεκτονικής ARM 32-bit με υποστήριξη στο EABI5. Επίσης, βλέπουμε ότι είναι dynamically linked, το path του interpreter, την έκδοση του πυρήνα αλλά και ότι είναι not stripped, δηλαδή περιέχει πληροφορίες αποσφαλμάτωσης. Από τα παραπάνω επίσης επιβεβαιώνουμε ότι το αρχείο είναι συμβατό με την αρχιτεκτονική του guest μηχανήματος στο QEMU.

Η δεύτερη εντολή μας δίνει πληροφορίες με περισσότερη λεπτομέρεια για το εκτελέσιμο αρχείο. Με τις σημαίες -h (--file-header) και -A (--arch-specific) που χρησιμοποιούμε με την εντολή, ζητάμε όσες πληροφορίες περιέχονται στο ELF header του αρχείου και όσες αναφέρονται στην αρχιτεκτονική του. Οι πιο βασικές αφορούν την υποστήριξη hard-float ABI, την χρήση VFP αναφορικά με την εκτέλεση των FLOPs, την ISA και την target αρχιτεκτονική, δηλαδή τα χαρακτηριστικά που έχουμε επιλέξει για το guest μηχάνημα.

5. Χρησιμοποιώντας τον cross compiler του linaro κάνουμε compile τον κώδικα του αρχείου phods.c τρέχοντας την παρακάτω εντολή:

```
$HOME/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc-4.8.3 -std=c99 phods.c -O0 -Wall -o phods_linaro.out
```

Η παραπάνω εντολή όμως απαιτεί να εγκαταστήσουμε πρώτα το παρακάτω πακέτο:

```
sudo apt-get install lib32z1-dev
```

Και μετά από την εκτέλεση της εντολής ls -lh παίρνουμε την παρακάτω έξοδο:

```
-rwxr-xr-x  1 jim jim  15K Jan  8 08:35 phods_crosstool.out
-rwxr-xr-x  1 jim jim 8.7K Jan  8 08:35 phods_linaro.out
```

Παρατηρούμε ότι το εκτελέσιμο που παράγει ο custom cross compiler του crosstool-ng έχει μέγεθος 15KB, ενώ το εκτελέσιμο που παράγεται με τον compiler του linaro έχει μέγεθος 8.7KB. Εκτελώντας πάλι την ίδια εντολή με πριν παίρνουμε την εξής έξοδο:

```
ldd -v ~/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc
```

```
jim@Dimitris:~$ ldd -v ~/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc

        linux-gate.so.1 (0xf7ef9000)
        libstdc++.so.6 => /lib32/libstdc++.so.6 (0xf7cc3000)
        libm.so.6 => /lib32/libm.so.6 (0xf7bbe000)
        libpthread.so.0 => /lib32/libpthread.so.0 (0xf7bb9000)
        libdl.so.2 => /lib32/libdl.so.2 (0xf7bb4000)
        libgcc_s.so.1 => /lib32/libgcc_s.so.1 (0xf7b8d000)
        libc.so.6 => /lib32/libc.so.6 (0xf7965000)
        /lib/ld-linux.so.2 (0xf7efb000)

        Version information:
        /home/jim/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc:
                libstdc++.so.6 (GLIBCXX_3.4) => /lib32/libstdc++.so.6
                libc.so.6 (GLIBC_2.3) => /lib32/libc.so.6
                libc.so.6 (GLIBC_2.2) => /lib32/libc.so.6
                libc.so.6 (GLIBC_2.1) => /lib32/libc.so.6
                libc.so.6 (GLIBC_2.1.3) => /lib32/libc.so.6
                libc.so.6 (GLIBC_2.0) => /lib32/libc.so.6
```

Παρατηρούμε ότι και ο compiler του linaro χρησιμοποιεί την βιβλιοθήκη glibc, με μόνη διαφορά ότι έχει την έκδοση των 32 bit. Επομένως, είναι απολύτως λογικό το εκτελέσιμο που παράγεται να έχει σημαντικά μικρότερο μέγεθος.

6. Εκτελώντας την παρακάτω εντολή παίρνουμε την αντίστοιχη έξοδο:

```
file phods_linaro.out
```

```
jim@Dimitris:~$ file phods_linaro.out
phods_linaro.out: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter
/lib/ld-linux-armhf.so.3, for GNU/Linux 3.1.1, BuildID[sha1]=d569a5c14b774d8733477cfb1b9a4316785ce667, not
 stripped
```

Συμπεραίνουμε λοιπόν ότι και τα δύο εκτελέσιμα αρχεία πρόκειται για αρχεία ELF των 32 bit. Παρά το γεγονός ότι χρησιμοποιούνται διαφορετικές εκδόσεις της βιβλιοθήκης glibc, 64 bit και 32 bit, και τα δύο εκτελέσιμα προορίζεται να τρέξουν σε guest μηχάνημα αρχιτεκτονικής 32 bit. Επομένως, δεν θα υπάρξει κανένα πρόβλημα κατά την εκτέλεση.

7. Κάνουμε ξανά compile το αρχείο phods.c με τους δύο cross-compilers, αλλά αυτή την φορά βάζουμε επιπλέον το flag -static, το οποίο κάνει static linking την βιβλιοθήκη της C. Έτσι, τρέχοντας ξανά την εντολή ls -lh παίρνουμε την παρακάτω έξοδο:

```
-rwxr-xr-x  1 jim jim 2.5M Jan  8 08:57 phods_crosstool.out
-rwxr-xr-x  1 jim jim 497K Jan  8 08:57 phods_linaro.out
```

Παρατηρούμε ότι το μέγεθος και των δύο εκτελέσιμων αρχείων είναι πολύ μεγαλύτερο σε σχέση με το προηγούμενο compile. Αυτό είναι λογικό, αφού ο cross-compiler κατά το static linking ενσωματώνει όλες τις βιβλιοθήκες της C που κάνει reference το πρόγραμμα phods. Αυτό έχει ως αποτέλεσμα οι βιβλιοθήκες να συμπεριλαμβάνονται στα τελικά εκτελέσιμα αρχεία, γεγονός που αυξάνει δραματικά το μέγεθός τους.

8. Έστω ότι προσθέτουμε μία δική μας συνάρτηση mlab_foo() στη glibc και δημιουργούμε έναν cross compiler με τον crosstool-ng που κάνει χρήση της ανανεωμένης glibc. Έπειτα, δημιουργούμε ένα αρχείο my_foo.c στο οποίο κάνουμε χρήση της νέας συνάρτησης που δημιουργήσαμε και το κάνουμε cross compile με flags -Wall -O0 -o my_foo.out.

   A. Αν προσπαθήσουμε να εκτελέσουμε το my_foo.out στο host μηχάνημα, αυτό δεν θα εκτελεστεί, καθώς το εκτελέσιμο προορίζεται για την αρχιτεκτονική του target μηχανήματος.

   B. Αν προσπαθήσουμε να εκτελέσουμε το my_foo.out στο guest μηχάνημα, αυτό δεν θα εκτελεστεί, καθώς δεν υπάρχει η ανανεωμένη glibc για να την κάνει dynamically link στο εκτελέσιμο.

   C. Προσθέτοντας το flag -static και κάνοντας ξανά compile το αρχείο, παρατηρούμε ότι αυτό εκτελείται. Τώρα, λόγω του static linking, η ανανεωμένη glibc θα ενσωματωθεί στο εκτελέσιμο κατά το compilation και όχι κατά την διάρκεια του runtime, όπως θα γινόταν με το dynamic linking.

## ***Άσκηση 2:***

Στο δεύτερο μέρος θα περιγράψουμε τα βήματα που ακολουθήθηκαν για τη δημιουργία ενός νέου πυρήνα Linux για αρχιτεκτονική ARM μέσω cross-compiling, καθώς και την ενσωμάτωση ενός νέου system call στον πυρήνα. Παρουσιάζονται λεπτομερώς τα προβλήματα που προέκυψαν και οι λύσεις τους.

1. Αρχικά θα χρησιμοποιήσουμε τον cross compiler που χτίσαμε με το crosstool-ng, προκειμένου να χτίσουμε έναν νέο πυρήνα για το Debian OS.

2. Αρχικοποιούμε τη διαδικασία αντιγράφοντας το directory **/boot/** από το Debian στο QEMU στο host μηχάνημα. Με αυτόν τον τρόπο μπορούμε να συγκρίνουμε τα υπάρχοντα αρχεία του Linux image και του initrd με εκείνα που θα παραχθούν μετά την εγκατάσταση του νέου πυρήνα.

```
scp -P 22223 -r root@localhost:/boot ./
```

3. Χρειαζόμαστε τον πηγαίο κώδικα του πυρήνα που θα χρησιμοποιήσουμε. Επομένως, θα κατεβάσουμε τον πηγαίο κώδικα που παρέχεται από τη **Debian**, εκτελώντας τις παρακάτω εντολές στο guest μηχάνημα:

```
root@localhost:~$ apt-get update
root@localhost:~$ apt-get install linux-source
```

Η εκτέλεση αυτή κατεβάζει στο directory `/usr/src` το αρχείο `linux-source-3.16.tar.xz`

4. Το compiling του πυρήνα θα γίνει στο host μηχάνημα γιατί ο χρόνος που χρεάζεται για να γίνει στο guest μηχάνημα είναι απαγορευτικός. Αντιγράφουμε λοιπόν στο host μηχάνημα το αρχείο που κατεβάσαμε και το κάνουμε extract.

```
scp -P 22223 -r root@localhost:/usr/src/linux-source-3.16.tar.xz
./Documents/embedded
cd ~/Documents/embedded
tar -xf linux-source-3.16.tar.xz
```

5. Επειδή κάνουμε cross-compiling πρέπει να υποδείξουμε στο make την target αρχιτεκτονική και τον cross compiler που θα χρησιμοποιήσουμε. Ο ευκολότερος τρόπος είναι να εκτελέσουμε την εντολή ως εξής:

```
zoegk@ubuntu:~/Documents/embedded/linux-source-3.16 make ARCH=arm
CROSS_COMPILE=$HOME/x-tools/arm-cortexa9_neon-linux-
gnueabihf/bin/arm-cortexa9_neon-linux-gnueabihf- makeconfig
```

Αφού επιλέξουμε τις ρυθμίσεις που επιθυμούμε, κάνουμε Save και οι ρυθμίσεις αποθηκεύονται σε ένα αρχείο στο untarred directory του linux-source με όνομα `.config`. Στο πλαίσιο της άσκησης, μας παρέχεται ένα προκαθορισμένο αρχείο διαμόρφωσης πυρήνα (**kernel_config**), το οποίο βρίσκεται στο **helios**. Για να το χρησιμοποιήσουμε κατεβάζουμε το αρχείο στον υπολογιστή μας. Στην συνέχεια τοποθετούμε το στο directory του πηγαίου κώδικα του πυρήνα ως `.config`:

```
zoegk@ubuntu:~/Documents/embedded/linux-source-3.16$ cp
/Documents/embedded/Lab6/kernel_config .config
```

Κατεβάσαμε, επιπλέον, από το **helios** το αρχείο builddeb_hf.patch με το οποίο κάναμε patch το script scripts/package/builddeb που περιλαμβάνεται στον πηγαίο κώδικα που κατεβάσαμε. Επειδή η ίδια η εντολή patch δεν λειτουργούσε κάναμε τις αλλαγές χειροκίνητα.

Στη συνέχεια, είμαστε έτοιμοι να χτίσουμε τον πυρήνα του linux εκτελώντας την παρακάτω εντολή:

```
zoegk@ubuntu:~/Documents/embedded/linux-source-3.16 make ARCH=arm
CROSS_COMPILE=$HOME/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-
cortexa9_neon-linux-gnueabihf-
```

Όπου επειδή κάνουμε cross-compiling, υποδείξαμε στο make την target αρχιτεκτονική και τον cross compiler που θα χρησιμοποιήσουμε.

Κατά την εκτέλεση της εντολής, προέκυψαν διάφορα Assembler messages που υποδείκνυαν unrecognized character '#' σε διάφορα .S αρχεία.

Συγκεκριμένα η γραμμή σε όλα τα αρχεία ήταν η εξής:

```
.section ".proc.info.init", #alloc, #execinstr
```

Το πρόβλημα οφείλεται στο ότι στην αρχιτεκτονική ARM, το `#` στο πλαίσιο `.section` μπορεί να προκαλέσει ασυμβατότητα αν η σύνταξη δεν υποστηρίζεται από την έκδοση του assembler ή από το συγκεκριμένο cross-compiler.

Η λύση ήταν να τροποποιηθεί η γραμμή σε:

```
.section ".proc.info.init", "ax"
```

Η αλλαγή έγινε επειδή ο συνδυασμός `"ax"` είναι ορθός και αναγνωρίζεται από τον assembler ως χαρακτηρισμός για section που είναι **allocatable** (`a`) και **executable** (`x`).

Η αντικατάσταση πραγματοποιήθηκε σε όλα τα σχετικά αρχεία με την παρακάτω εντολή:

```
find ./ -type f -exec sed -i 's/\.section ".proc.info.init", #alloc,
#execinstr/\.section ".proc.info.init", "ax"/g' {} +
```

- `find ./ -type f`: Εντοπίζει όλα τα αρχεία στον τρέχοντα φάκελο και τους υποκαταλόγους του.
- `-exec sed -i`: Εκτελεί την εντολή `sed` για κάθε αρχείο, τροποποιώντας το περιεχόμενο στη θέση του.
- `'s/\.section ".proc.info.init", #alloc, #execinstr/\.section ".proc.info.init", "ax"/g'`:
- Το `s/παλιό/νέο/g` είναι η εντολή αντικατάστασης της `sed`.
- Αντικαθιστά την παλιά γραμμή `.section ".proc.info.init", #alloc, #execinstr` με τη νέα `.section ".proc.info.init", "ax"`.
- `{} +`: Επιτρέπει στη `find` να περάσει πολλά αρχεία ταυτόχρονα στη `sed`.

Μετά την αντικατάσταση το σφάλμα επιλύθηκε και προχωρήσαμε στο επόμενο βήμα.

6. Προκειμένου να δημιουργήσουμε τα 3 .deb πακέτα που θα ανέβουν στο qemu, εκτελέσαμε την ακόλουθη εντολή (με -j 4 για να χρησιμοποιηθούν όλοι οι διαθέσιμοι πυρήνες του υπολογιστή):

```
zoegk@ubuntu:~/Documents/embedded/linux-source-3.16 make ARCH=arm
CROSS_COMPILE=$HOME/x-tools/arm-cortexa9_neon-linux-gnueabihf/bin/arm-
cortexa9_neon-linux-gnueabihf- deb-pkg -j4
```

Τα παραγόμενα `.deb` αρχεία περιείχαν τα ακόλουθα αρχεία:

- `debian-binary`
- `control.tar.zst`
- `data.tar.zst`

Τα `.deb` πακέτα περιέχουν τρία βασικά αρχεία, καθένα με συγκεκριμένη λειτουργία. Το αρχείο `debian-binary` είναι ένα μικρό αρχείο κειμένου που περιέχει την έκδοση του Debian package format που χρησιμοποιείται. Χρησιμοποιείται από το `dpkg` για να διασφαλίσει ότι το πακέτο είναι συμβατό με την έκδοση του διαχειριστή πακέτων. Αν, για παράδειγμα, το περιεχόμενο είναι `2.0`, σημαίνει ότι το πακέτο ακολουθεί το Debian Package Format version 2.0. Το αρχείο `control.tar.*` περιλαμβάνει τα metadata του πακέτου, όπως βασικές πληροφορίες (π.χ., όνομα, έκδοση, εξαρτήσεις) και scripts που εκτελούνται κατά την εγκατάσταση ή απεγκατάσταση (π.χ., `postinst, prerm`). Αυτές οι πληροφορίες καθορίζουν πώς θα εγκατασταθεί, διαμορφωθεί ή αφαιρεθεί το πακέτο. Τέλος, το αρχείο `data.tar.*` περιέχει τα πραγματικά δεδομένα του πακέτου, όπως εκτελέσιμα αρχεία, βιβλιοθήκες και configuration files. Κατά την εγκατάσταση, το περιεχόμενο αυτού του αρχείου αποσυμπιέζεται και τοποθετείται στα αντίστοιχα directories του συστήματος, όπως `/usr/bin` ή `/etc`.

Το πρόβλημα προέκυψε επειδή το λειτουργικό σύστημα Debian 8 που χρησιμοποιείται στο QEMU δεν υποστηρίζει **Zstandard (Zst)** compression, το οποίο χρησιμοποιείται από νεότερες εκδόσεις του εργαλείου `dpkg`.

Η επίλυση του προβλήματος έγινε με τη μετατροπή των αρχείων `control.tar.zst` και `data.tar.zst` σε μορφή **xz**, που υποστηρίζεται από το Debian 8.

Ακολουθήθηκαν τα παρακάτω βήματα:

- **Δημιουργία και Είσοδος σε Εργασιακό Φάκελο**: Η χρήση του εργασιακού φακέλου ήταν απαραίτητη για να διατηρηθούν τα metadata των αρχείων ανέπαφα:

```
mkdir -p linux-libc-dev-workdir
   cd linux-libc-dev-workdir
```

- Χρησιμοποιήθηκε η εντολή `ar` για την εξαγωγή των περιεχομένων του `.deb`:
  Τα εξαγόμενα αρχεία περιλαμβάνουν:
  - `debian-binary`: Πληροφορίες για τη μορφή του πακέτου.
  - `control.tar.zst`: Metadata και scripts εγκατάστασης.
  - `data.tar.zst`: Τα δεδομένα του πακέτου.

- Αποσυμπίεση και επανασυμπίεση σε υποστηριζόμενη μορφή:

```
unzstd control.tar.zst

xz control.tar

unzstd data.tar.zst

xz data.tar
```

- Καθαρισμός Παλιών Αρχείων `.zst`:

```
rm control.tar.zst data.tar.zst
```

- Το πακέτο επανασυσκευάστηκε χρησιμοποιώντας τη μορφή `.xz`:

```
ar rcs fixed_linux-libc-dev_3.16.84-18_armhf.deb debian-binary
control.tar.xz data.tar.xz
```

- Επιστροφή στον αρχικό φάκελο και διαγραφή του εργασιακού φακέλου:

```
mv fixed_linux-libc-dev_3.16.84-18_armhf.deb ../

cd ..

rm -rf linux-libc-dev-workdir
```

Επαναλαμβάνοντας αυτή την διαδικασία για όλα τα deb αρχεία συνεχίσαμε στο επόμενο βήμα.

7. Ανεβάζουμε τα αρχεία στο target μηχάνημα με την εντολή:

```
scp -P 22223 fixed_linux-image-3.16.84_3.16.84-18_armhf.deb
root@localhost:/root/

scp -P 22223 fixed_linux-headers-3.16.84_3.16.84-18_armhf.deb
root@localhost:/root/
```

```
scp -P 22223 fixed_linux-libc-dev_3.16.84-18_armhf.deb
root@localhost:/root/
```

Και κάνουμε την εγκατάσταση με την εντολή:

```
dpkg -i /root/fixed_linux-image-3.16.84_3.16.84-18_armhf.deb
dpkg -i /root/fixed_linux-headers-3.16.84_3.16.84-18_armhf.deb
dpkg -i /root/fixed_linux-libc-dev_3.16.84-18_armhf.deb
```

8. Ελέγχουμε το /boot directory και βλέπουμε ότι έχουν προστεθεί τα καινούργια αρχεία `initrd.img-3.16.84` και `vmlinuz-3.16.84`

```
root@debian-armhf:~# ls /boot
config-3.16.84                System.map-3.16.84
config-3.2.0-4-vexpress       System.map-3.2.0-4-vexpress
initrd.img                    vmlinuz
initrd.img-3.16.84            vmlinuz-3.16.84
initrd.img-3.2.0-4-vexpress   vmlinuz-3.2.0-4-vexpress
```

Τα αρχεία `vmlinuz-3.16.84` και `initrd.img-3.16.84` είναι ο συμπιεσμένος πυρήνας του Linux και η προσωρινή εικόνα δίσκου που προετοιμάζει το περιβάλλον εκκίνησης φορτώνοντας απαραίτητα modules, ώστε ο πυρήνας να αποκτήσει πρόσβαση στο πραγματικό σύστημα αρχείων και να συνεχίσει την εκκίνηση.

Μεταφέρουμε όλο τον φάκελο /boot στο host μηχάνημα, και κάνουμε reboot το qemu.

Εκκινούμε πλέον με τα καινούργια μας αρχεία τον qemu:

```
sudo qemu-system-arm -M vexpress-a9 \
-kernel vmlinuz-3.16.84 \
-initrd initrd.img-3.16.84 \
-drive if=sd,file=debian_wheezy_armhf_standard.qcow2 \
-append "root=/dev/mmcblk0p2" \
-net nic -net user,hostfwd=tcp:127.0.0.1:22223-:22
```

*Ερώτημα 1ο*

Παρακάτω, φαίνονται τα αποτελέσματα της εντολής uname -a, πριν και μετά την εγκατάσταση του νέου πυρήνα:

```
root@debian-armhf:/usr/src# uname -a
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1 armv7l GNU
/Linux
```

```
root@debian-armhf:~# uname -a
Linux debian-armhf 3.16.84 #16 SMP Wed Jan 8 22:30:57 EET 2025 armv7l GNU/Linux
root@debian-armhf:~#
```

Η εντολή `uname -a` χρησιμοποιείται στο Linux για να εμφανίσει πληροφορίες σχετικά με το λειτουργικό σύστημα και τον πυρήνα που χρησιμοποιείται. Περιλαμβάνει σημαντικά στοιχεία όπως το όνομα του συστήματος (π.χ., Linux), το όνομα του μηχανήματος (hostname), την έκδοση και το όνομα του πυρήνα, την ημερομηνία και ώρα κατασκευής του, καθώς και την υποστηριζόμενη αρχιτεκτονική του συστήματος (π.χ., `armv7l`). Αυτή η πληροφορία είναι ζωτικής σημασίας όταν πραγματοποιούμε αλλαγές στον πυρήνα, καθώς μας επιτρέπει να επιβεβαιώσουμε ότι ο νέος πυρήνας έχει εγκατασταθεί και χρησιμοποιείται. Βλέπουμε πως ο νέος πυρήνας έχει εγκατασταθεί σωστά και χρησιμοποιείται τη δεύτερη φορά που ανοίγουμε το vm.

Στα πλαίσια του cross-compiling, η εντολή αυτή μας βοηθά να επαληθεύσουμε ότι ο νέος πυρήνας που χτίσαμε στο **host μηχάνημα** (με διαφορετική αρχιτεκτονική) εγκαταστάθηκε επιτυχώς στο **target μηχάνημα** (QEMU).

Μετά την επιτυχή εγκατάσταση και εκκίνηση με τον νέο πυρήνα, η εντολή εμφάνισε την έκδοση `3.16.84`, επιβεβαιώνοντας ότι η διαδικασία cross-compiling και εγκατάστασης ολοκληρώθηκε σωστά.

*Ερώτημα 2ο*

Στα δύο τελευταία ερωτήματα ζητείται να υλοποιήσουμε ένα custom system call που θα χρησιμοποιεί τη συνάρτηση `printk` για να καταγράψει στο log του πυρήνα τη φράση: **“Greeting from kernel and team no %d”**, συνοδευόμενη από το όνομα της ομάδας μας - **3**. Για την υλοποίηση αυτού του στόχου, απαιτείται η δημιουργία ενός αρχείου πηγαίου κώδικα με την ονομασία `my_sys_call.c`, το οποίο θα τοποθετηθεί στο root directory του λειτουργικού συστήματος. Στο αρχείο αυτό θα γράψουμε τον κώδικα που θα υλοποιεί τη λειτουργία του system call.

```c
#include <linux/kernel.h>

asmlinkage long my_sys_call(void){
    int team = 3;
    printk(KERN_ALERT "Greetings from kernel and team no. %d\n",
team);
    return 0;
}
```

Για τον παραπάνω κώδικα:

- `#include <linux/kernel.h>`: Αυτό το header αρχείο περιλαμβάνει βασικές δηλώσεις και macros που σχετίζονται με λειτουργίες του πυρήνα, όπως η `printk`.
- `asmlinkage long sys_greet(void)`: Η δήλωση του system call. Η λέξη-κλειδί `asmlinkage` διασφαλίζει ότι τα ορίσματα της συνάρτησης περνούν μέσω του stack, κάτι που είναι απαραίτητο για system calls. Η συνάρτηση επιστρέφει έναν ακέραιο τύπο `long`.
- Τοπική μεταβλητή `int team = 3;`: Η μεταβλητή `team` δηλώνεται και αρχικοποιείται με την τιμή `3`, που αντιπροσωπεύει τον αριθμό της ομάδας.
- `printk(KERN_ALERT "...", team);`: Η συνάρτηση `printk` χρησιμοποιείται για την καταγραφή μηνυμάτων στο log του πυρήνα. Το επίπεδο `KERN_ALERT` υποδεικνύει ότι πρόκειται για ένα μήνυμα υψηλής προτεραιότητας. Το μήνυμα περιλαμβάνει τη φράση "Greetings from kernel and team no." και τον αριθμό της ομάδας που περνιέται μέσω της μεταβλητής `team`.
- `return 0;`: Η συνάρτηση επιστρέφει το 0 για να υποδηλώσει επιτυχή εκτέλεση.

Αρχικά, στο host μηχάνημα, αποθηκεύουμε το αρχείο μας `my_sys_call.c` στο `linux-source-3.16/kernel/my_sys_call.c`.

Υλοποιούμε στο `linux-source-3.16/kernel/Makefile` το οποίο δηλώνει ότι το αρχείο `my_sys_call.c` πρέπει να συμπεριληφθεί κατά τη μεταγλώττιση.:

```
obj-y := my_sys_call.o
```

Για να υλοποιήσουμε το custom system call που δημιουργήσαμε, ακολουθήσαμε μια σειρά από βήματα που περιλάμβαναν την προσθήκη και ρύθμιση των απαραίτητων αρχείων και καταχωρήσεων στον πηγαίο κώδικα του πυρήνα.

Για να γίνει η λειτουργία αυτή διαθέσιμη, έπρεπε να δηλωθεί και να ενσωματωθεί στο system call table. Αρχικά, στο αρχείο `linux-source-3.16/include/linux/syscalls.h` , για να είναι ορατή η νέα συνάρτηση στον πηγαίο κώδικα, προσθέσαμε τη δήλωση της συνάρτησης:

```
asmlinkage long my_sys_call(void);
```

Στη συνέχεια, εντοπίσαμε το αρχείο `linux-source-3.16/arch/arm/kernel/calls.S` και, βλέποντας ότι το τελευταίο καταχωρημένο system call ήταν το 385, προσθέσαμε στο τέλος του αρχείου τη γραμμή, για να καταχωρηθεί η νέα λειτουργία στον πίνακα system calls για την αρχιτεκτονική ARM.

```
/* 386 */ CALL(my_sys_call)
```

Παράλληλα, εισάγουμε τον αριθμό αναφοράς (`syscall number`) για το νέο system call στο `linux-source-3.16/arch/arm/include/asm/unistd.h` προσθέτοντας:

```
#define __NR_my_sys_call (__NR_SYSCALL_BASE+386)
```

Επίσης, στο αρχείο `linux-source-3.16/arch/x86/syscalls/syscall_32.tbl` εισάγαμε την καταχώρηση, για να ενεργοποιηθεί το system call για την αρχιτεκτονική x86.

```
386         i386            my_sys_call           my_sys_call
```

Έπειτα, προσθέσαμε στο κύριο Makefile του πυρήνα μια γραμμή, ώστε να συμπεριληφθεί κατά τη μεταγλώττιση. Η γραμμή που προστέθηκε ήταν:

```
core-y += kernel/ mm/ fs/ ipc/ security/ crypto/ block/ greet/
```

Η παραπάνω γραμμή, προσθέτει τον φάκελο `kernel/` στη διαδικασία μεταγλώττισης.

Τέλος, αφού ολοκληρώσαμε τις παραπάνω ρυθμίσεις, μεταγλωττίσαμε τον πυρήνα με τις νέες ρυθμίσεις. Η διαδικασία μεταγλώττισης ολοκληρώθηκε χωρίς προβλήματα. Χρησιμοποιήσαμε το παραγόμενο image για να ανοίξουμε το VM, επιβεβαιώνοντας ότι το νέο system call λειτουργεί όπως αναμενόταν.

*Ερώτημα 3ο*

Για να δοκιμάσουμε το custom system call που υλοποιήσαμε, ακολουθήσαμε τα παρακάτω βήματα:

Γράψαμε το πρόγραμμα `test.c`, το οποίο καλεί το system call μας. Ο κώδικας του αρχείου είναι ο εξής:

```
#include <unistd.h>
#include <sys/syscall.h>
#include <stdio.h>
#include <errno.h>

#define SYSCALL_NR 386

int main(void) {
    printf("Attempting to invoke the 'greet' system call...\n");

    long ret = syscall(SYSCALL_NR);

    if (ret == 0) {
        printf("System call executed successfully! Return value:
%ld\n", ret);
```

```
    } else {
        printf("System call failed. Error: %s (Code: %d)\n",
strerror(errno), errno);
    }

    printf("Exiting test program.\n");
    return 0;
}
```

Στον κώδικα, περιλαμβάνονται τα απαραίτητα headers:

- `unistd.h` για το σύστημα κλήσεων.
- `sys/syscall.h` για τη χρήση του `syscall` wrapper.
- `stdio.h` για την έξοδο στην κονσόλα.

Ορίζουμε τη μακροεντολή `SYSCALL_NR` με την τιμή `386`, η οποία αντιστοιχεί στο index του system call που προσθέσαμε.

Μεταφέραμε το αρχείο `test.c` στο VM μέσω QEMU και το μεταγλωττίσαμε με την εντολή:

```
gcc -o test.out test.c
```

Στη συνέχεια, εκτελέσαμε το παραγόμενο εκτελέσιμο και η έξοδος ήταν η αναμενόμενη:

```
./test.out
```

Για να επιβεβαιώσουμε ότι το system call εκτέλεσε την αναμενόμενη λειτουργία, χρησιμοποιούμε την εντολή:

```
dmesg | tail
```

```
root@debian-armhf:~# ls
fixed_linux-headers-3.16.84_3.16.84-18_armhf.deb  fixed_linux-libc-dev_3.16.84-18_armhf.deb  test.c
fixed_linux-image-3.16.84_3.16.84-18_armhf.deb    legacy
root@debian-armhf:~# gcc -o test.out test.c
root@debian-armhf:~# ./test.out
Attempting to invoke our system call...
System call executed succesfully! Return value:0
Exiting test program.
root@debian-armhf:~# dmesg | tail
[  105.866961] IPv6: ADDRCONF(NETDEV_UP): eth0: link is not ready
[  107.497250] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
[  120.848019] RPC: Registered named UNIX socket transport module.
[  120.855278] RPC: Registered udp transport module.
[  120.876857] RPC: Registered tcp transport module.
[  120.887183] RPC: Registered tcp NFSv4.1 backchannel transport module.
[  121.268995] FS-Cache: Loaded
[  122.030002] FS-Cache: Netfs 'nfs' registered for caching
[  123.011115] Installing knfsd (copyright (C) 1996 okir@monad.swb.de).
[  406.957993] Greetings from kernel and team no. 3
root@debian-armhf:~#
```